

THE AGENT WHO AVOIDED THE ORDINARY

In this lab, you will implement the simplest agent that learns to predict the outcome of its actions and tries another action when it gets bored.

Learning objective

Upon completing this lab, you will be able to implement artificial agents based on the 'conceptual inversion of the interaction cycle.' In this framework, the agent starts by taking action and then receives a sensory signal which is an outcome of action. This contrasts with traditional AI agents, which first perceive their environment before deciding how to act.

Setup

Define the Agent class

```
In [2]: class Agent:
    def __init__(self):
        """ Creating our agent """
        self._action = None
        self._predicted_outcome = None

    def action(self, _outcome):
        """ tracing the previous cycle """
        if self._action is not None:
            print(f"Action: {self._action}, Prediction: {self._predicted_outcome}, Outcome: {
                f"Satisfaction: {self._predicted_outcome == _outcome}")

        """ Computing the next action to enact """
        # TODO: Implement the agent's decision mechanism
        self._action = 0
        # TODO: Implement the agent's anticipation mechanism
        self._predicted_outcome = 0
        return self._action
```

Environment1 class

```
In [3]: class Environment1:
    """ In Environment 1, action 0 yields outcome 0, action 1 yields outcome 1 """
    def outcome(self, _action):
        # return int(input("entre 0 1 ou 2"))
        if _action == 0:
            return 0
        else:
            return 1
```

Environment2 class

```
In [4]: class Environment2:
        """ In Environment 2, action 0 yields outcome 1, action 1 yields outcome 0 """
        def outcome(self, _action):
            if _action == 0:
                return 1
            else:
                return 0
```

Instantiate the agent

```
In [5]: a = Agent()
```

Instantiate the environment

```
In [6]: e = Environment1()
```

Test run the simulation

```
In [7]: outcome = 0
        for i in range(10):
            action = a.action(outcome)
            outcome = e.outcome(action)
```

```
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
```

Observe that, on each interaction cycle, the agent correctly predicts the outcomes. The agent's satisfaction is True because its predictions are correct.

PRELIMINARY EXERCISE

Run the agent in Environment2. Observe that its satisfaction becomes False. This agent is not satisfied in Environment2!

Now you see the goal of this assignment: design an agent that learns to be satisfied when it is run either in Environment1 or in Environment2.

ASSIGNMENT

Implement Agent1 that:

- learns to predict the outcome of its actions
- chooses a different action when its predictions have been correct for 4 times in a row

The agent can choose two possible actions `0` or `1`, and can receive two possible outcomes: `0` or `1`.

It computes the prediction on the assumption that the same action always yields the same outcome in a given environment. You must thus implement a memory of the obtained outcomes for each action.

Create your own agent by overriding the class Agent

Create an agent that learns to correctly predict the outcome of its actions in both Environment1 and Environment2.

You may add any class attribute or method you deem useful.

```
In [11]: class Agent1(Agent):
    def __init__(self):
        super().__init__()
        # je garde en mémoire ce que j'ai obtenu la dernière fois pour chaque action
        self.souvenirs = {}
        # je compte combien de fois de suite j'ai deviné juste
        self.compteur_ennui = 0

    def action(self, _outcome):
        # si ce n'est pas le tout premier cycle, on regarde ce qui vient de se passer
        if self._action is not None:
            j_ai_bien_predit = (self._predicted_outcome == _outcome)
            print(f"Action: {self._action}, Prediction: {self._predicted_outcome}, Outcome: {
                f"Satisfaction: {j_ai_bien_predit}")

            # on retient ce qu'on vient d'apprendre
            self.souvenirs[self._action] = _outcome

            # si on a bien deviné, on s'ennuie un peu plus, sinon on repart à zéro
            if j_ai_bien_predit:
                self.compteur_ennui += 1
            else:
                self.compteur_ennui = 0

        # premier cycle : on commence par l'action 0
        if self._action is None:
            self._action = 0
        # si on s'ennuie depuis 4 cycles, on change d'action
        elif self.compteur_ennui >= 4:
            self._action = 1 - self._action # bascule entre 0 et 1
            self.compteur_ennui = 0

        # on prédit selon ce qu'on a déjà vu pour cette action (0 si jamais essayée)
        self._predicted_outcome = self.souvenirs.get(self._action, 0)

        return self._action
```

Test your agent in Environment1

```
In [12]: a = Agent1()
e = Environment1()
outcome = 0
for i in range(20):
    action = a.action(outcome)
    outcome = e.outcome(action)
```

```

Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 1, Satisfaction: False
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 0, Outcome: 0, Satisfaction: True

```

Test your agent in Environment2

```

In [14]: a = Agent1()
         e = Environment2()
         outcome = 0
         for i in range(20):
             action = a.action(outcome)
             outcome = e.outcome(action)

```

```

Action: 0, Prediction: 0, Outcome: 1, Satisfaction: False
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 1, Prediction: 0, Outcome: 0, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True
Action: 0, Prediction: 1, Outcome: 1, Satisfaction: True

```

Report

Explain what you programmed and what results you observed. Export this document as PDF including your code, the traces you obtained, and your explanations below (no more than a few paragraphs):

Implémentation

J'ai implémenté Agent1 en ajoutant deux éléments à la classe de base : une mémoire des outcomes observés pour chaque action, et un compteur d'ennui.

La mémoire est un dictionnaire qui associe à chaque action le dernier outcome obtenu avec cette action. À chaque cycle, ce dictionnaire est mis à jour avec l'outcome qui vient d'être observé. La prédiction pour

le cycle suivant repose sur l'hypothèse qu'une même action produit toujours le même outcome dans un environnement donné : l'agent prédit donc le dernier outcome connu pour l'action choisie, ou 0 par défaut si cette action n'a encore jamais été essayée.

Le compteur d'ennui s'incrémente à chaque fois que la prédiction de l'agent est correcte, et se réinitialise à zéro dès qu'une prédiction est fausse. Lorsque ce compteur atteint 4, l'agent change d'action (il passe de 0 à 1 ou inversement) et remet le compteur à zéro. Cela force une exploration périodique, même lorsque l'agent prédit correctement depuis un moment.

Résultats observés

Dans Environment1 (où l'action 0 produit l'outcome 0, et l'action 1 produit l'outcome 1) : l'agent répète l'action 0 pendant 4 cycles avec une prédiction correcte à chaque fois, car sa prédiction par défaut (0) se trouve juste par coïncidence. Au 4e cycle, l'ennui est atteint. Au 5e cycle, l'agent bascule sur l'action 1, dont il n'a encore jamais observé l'outcome : sa prédiction (encore 0 par défaut) est donc fausse cette fois. À partir de ce moment, il a appris que l'action 1 produit l'outcome 1, et prédit correctement pendant 4 nouveaux cycles, avant de s'ennuyer à nouveau et de revenir à l'action 0. Ce cycle de 4 répétitions correctes suivies d'un changement d'action se répète ensuite indéfiniment.

Dans Environment2 (associations inversées : l'action 0 produit l'outcome 1, et l'action 1 produit l'outcome 0) : le comportement suit exactement la même logique, mais les associations apprises sont différentes. La toute première prédiction est fausse, car l'agent anticipe 0 par défaut pour l'action 0, alors que l'environnement renvoie 1. Après cette première erreur, l'agent mémorise la bonne association et ne se trompe plus jamais : seuls des changements volontaires d'action dus à l'ennui interrompent la répétition.

Interprétation

Ces deux traces montrent que l'agent ne code aucune règle fixe sur le monde : il apprend uniquement à partir de l'expérience accumulée dans sa mémoire, ce qui lui permet de s'adapter aussi bien à Environment1 qu'à Environment2, sans aucune modification de code entre les deux exécutions. Le mécanisme d'ennui garantit par ailleurs que l'agent n'est jamais figé : même quand ses prédictions sont parfaitement correctes, il continue périodiquement à explorer l'espace des actions.

In []: